Neurocomputing 174 (2016) 179–186

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Regression and classification using extreme learning machine based on L_1-norm and L_2-norm ☆

Xiong Luo *, Xiaohui Chang, Xiaojuan Ban

School of Computer and Communication Engineering, University of Science and Technology Beijing (USTB), Beijing 100083, China

ARTICLE INFO

Article history:
Received 28 September 2014
Received in revised form
27 February 2015
Accepted 1 March 2015
Available online 12 August 2015

Keywords:
Extreme learning machine
Ridge regression
Elastic net
Model selection
Bayesian information criterion (BIC)

ABSTRACT

Extreme learning machine (ELM) is a very simple machine learning algorithm and it can achieve a good generalization performance with extremely fast speed. Therefore it has practical significance for data analysis in real-world applications. However, it is implemented normally under the empirical risk minimization scheme and it may tend to generate a large-scale and over-fitting model. In this paper, an ELM model based on L_1-norm and L_2-norm regularizations is proposed to handle regression and multiple-class classification problems in a unified framework. The proposed model called L_1–L_2-ELM combines the grouping effect benefits of L_2 penalty and the tendency towards sparse solution of L_1 penalty, thus it can control the complexity of the network and prevent over-fitting. To solve the mixed penalty problem, the separate elastic net algorithm and Bayesian information criterion (BIC) are adopted to find the optimal model for each response variable. We test the L_1–L_2-ELM algorithm on one artificial case and nine benchmark data sets to evaluate its performance. Simulation results have shown that the proposed algorithms outperform the original ELM as well as other advanced ELM algorithms in terms of prediction accuracy, and it is more robust in both regression and classification applications.

© 2015 Elsevier B.V. All rights reserved.

1. Introduction

At the information stage, there has been a growing interest in the study of data analysis techniques. Techniques of data analysis can extract previously unknown, hidden, but potentially useful information and knowledge from original data, which is helpful to provide suggestions or decisions for future actions [1]. Single-hidden layer feedforward network (SLFN) is one of the classic methods used in data analysis due to its powerful nonlinear mapping capability [2]. However, it is clear that the learning speed of SLFN is far slower than required because of its slow gradient-based learning algorithm [3,4], which has imposed very challenging obstacles in practical applications.

Recently, extreme learning machine (ELM) for SLFNs was proposed by Huang et al. [5]. It randomly chooses the parameters of hidden nodes and analytically determines the output weights through the use of ordinary least square method. It tends to reach the solution straightforward without facing such trivial issues like local minima, improper learning rate, poor computational scalability, and so on. Compared with other traditional learning techniques, ELM can achieve a better generalization performance for classification and regression with extremely fast speed, which makes it work as an emergent technology for data analysis in many practical applications, such as wireless sensor networks [6]. However, basic ELM solutions may tend to generate an over-fitting model and are less stable in some situations [7]. Moreover, there is another question in the ELM design: how to obtain an appropriate neural network (NN) structure?

To overcome the problems ELM faced, several schemes have been proposed. A regularized ELM based on L_2 penalty was proposed by Deng et al. [8]. It avoids the generation of an over-fitting model and can provide more robust estimate as well as better generalization ability than ELM. But it cannot provide a suitable NN structure, which may lead to some issues while facing irrelevant variables. To find an appropriate NN structure with better robustness and generality, in [9], Rong et al. proposed a fast pruned ELM based on statistical tests for classification problems. Then using a similar statistical method that measures the relativity between the hidden nodes and the output nodes of ELM, Martínez-Martínez et al. proposed a regularized ELM based on L_1 penalty and hybrid penalties for regression problems in [10]. In [9,10], although those algorithms can generate a spare NN

☆This work was jointly supported by the National Natural Science Foundation of China under Grant nos. 61174103, 61272357, and 61004021, the National Key Technologies R&D Program of China under Grant no. 2015BAK38B01, the Aerospace Science Foundation of China under Grant no. 2014ZA74001, and the Fundamental Research Funds for Central Universities under Grant no. FRF-TP-11-002B.

* Corresponding author.
E-mail address: xluo@ustb.edu.cn (X. Luo).

http://dx.doi.org/10.1016/j.neucom.2015.03.112
0925-2312/© 2015 Elsevier B.V. All rights reserved.

structure, they do not provide a unified NN framework for both regression and classification problems. Miche et al. proposed an optimally pruned ELM called OP-ELM for regression and classification in [11]. Actually, it is also a regularized ELM through the use of the least angle regression (LARS) algorithm, i.e., a L_1 penalty. But this algorithm has its limitation while facing a group of high correlated variables. In fact, it just selects one variable from the group, which may lead to a suboptimal model finally.

In order to develop a more appropriate NN structure to overcome the hurdle that a suboptimal model faced, we propose a novel ELM algorithm based on L_1 penalty and L_2 penalty to deal with both multiple-output regression tasks and multiple-class classification tasks in a unified framework with the purpose of improving the robustness and generality of the model. Based on the L_1 norm, the L_2 penalty is also introduced to ELM for the purpose to encourage a group effect, then groups of correlated hidden nodes can be selected. Therefore, our proposed L_1–L_2-ELM benefits from ridge regression and the tendency towards sparse solution of the L_1 penalty, and it can generate a more suitable NN structure than OP-ELM. And elastic net algorithm is used to solve these mixed penalties [12]. Here, the multiple-class classification problem is transformed into a multiple-output regression problem. In general, elastic net is applied to deal with the special cases that have the single response variable. Moreover, it is obvious that different hidden nodes have varying degrees of relativity to the output nodes of ELM, some of hidden nodes may be weakly correlated with a certain output node, and some of other hidden nodes may be weakly correlated with another output node. Thus, for different output nodes, we may prune different hidden nodes, which can obtain a more appropriate NN structure. Considering the above two points, the separate elastic net algorithm and the Bayesian information criterion (BIC) [13] are adopted to find the optimal model for each response variable. Thus, the proposed algorithm may encourage a grouping effect and produce a sparse model with a higher predict accuracy, which tends to reduce over-fitting and provide a more robust model.

This paper is organized as follows. The SLFN based on ELM and classic regularization methods is analyzed in Section 2. The proposed ELM model based on L_1-norm and L_2-norm regularizations is presented in Section 3. The simulation results and discussion are provided in Section 4. The conclusion is summarized in Section 5.

2. Model description

2.1. ELM-based SLFN

Different from traditional theories that all the parameters of the feedforward NN need to be tuned to minimize the cost function, ELM theories claim that the hidden node learning parameters can be randomly assigned independently and the network output weights can be analytically determined by solving a linear system through the use of the least square method [14]. The training phase can be efficiently completed without time-consuming learning iterations and ELM can achieve a good generalization performance.

For P arbitrary distinct samples $(\mathbf{x}_i, \mathbf{t}_i)$ where $\mathbf{x}_i = [x_{i1}, x_{i2}, \ldots, x_{im}]^{\mathrm{T}} \in \mathbb{R}^m$ and $\mathbf{t}_i = [t_{i1}, t_{i2}, \ldots, t_{in}]^{\mathrm{T}} \in \mathbb{R}^n$, a standard SLFN with L hidden nodes can be mathematically modeled as

$$\mathbf{o}_i = \sum_{j=1}^{L} \beta_j G(\mathbf{a}_j, b_j, \mathbf{x}_i), \quad i = 1, 2, \ldots, P \tag{1}$$

where $\mathbf{a}_j$ and b_j are the learning parameters of hidden nodes, and $\beta_j = [\beta_{j1}, \beta_{j2}, \cdots, \beta_{jn}]^{\mathrm{T}}$ is the link connecting the jth hidden node to the output nodes, $G(\mathbf{a}_j, b_j, \mathbf{x}_i)$ is the output of the jth hidden node with respect to the input $\mathbf{x}_i$, $\mathbf{o}_i$ is the actual output of the neural network with respect to the input $\mathbf{x}_i$.

For additive hidden nodes with the sigmoid or threshold activation function $g(x)$: $\mathbb{R} \mapsto \mathbb{R}$, $G(\mathbf{a}_j, b_j, \mathbf{x}_i)$ is given by

$$G(\mathbf{a}_j, b_j, \mathbf{x}_i) = g(\mathbf{a}_j^{\mathrm{T}} \cdot \mathbf{x}_i + b_j), \quad \mathbf{a}_j \in \mathbb{R}^m, b_j \in \mathbb{R} \tag{2}$$

where $\mathbf{a}_j = [a_{j1}, a_{j2}, \ldots, a_{jm}]^{\mathrm{T}}$ is the weight vector connecting the jth hidden node and the input nodes, b_j is the threshold of the jth hidden node.

For radial basis function (RBF) hidden nodes with the gaussian or triangular activation function $g(x)$: $\mathbb{R} \mapsto \mathbb{R}$, $G(\mathbf{a}_j, b_j, \mathbf{x}_i)$ is given by

$$G(\mathbf{a}_j, b_j, \mathbf{x}_i) = g\left(b_j \| \mathbf{x}_i - \mathbf{a}_j \|_2\right), \quad \mathbf{a}_j \in \mathbb{R}^m, b_j \in \mathbb{R}^+ \tag{3}$$

where $\mathbf{a}_j$ and b_j are the center and impact factor of the jth RBF hidden node, $\mathbb{R}^+$ indicates the set of all positive real values. In addition, $\| \cdot \|_2$ denotes the L_2-norm. The SLFN with L hidden nodes can approximate these P samples with zero error, which means that the cost function $E = \sum_{i=1}^{P} \| \mathbf{o}_i - \mathbf{t}_i \|_2 = 0$, i.e., there exist $(\mathbf{a}_j, b_j)$ and β_j such that

$$\mathbf{t}_i = \sum_{j=1}^{L} \beta_j G(\mathbf{a}_j, b_j, \mathbf{x}_i), \quad i = 1, 2, \ldots, P \tag{4}$$

The above P equations can be rewritten compactly as

$$\mathbf{H}\beta = \mathbf{T} \tag{5}$$

where

$$\mathbf{H} = \begin{bmatrix} G(\mathbf{a}_1, b_1, \mathbf{x}_1) & \cdots & G(\mathbf{a}_L, b_L, \mathbf{x}_1) \\ \vdots & \ddots & \vdots \\ G(\mathbf{a}_1, b_1, \mathbf{x}_P) & \cdots & G(\mathbf{a}_L, b_L, \mathbf{x}_P) \end{bmatrix}_{P \times L},$$

$$\beta = \begin{bmatrix} \beta_1^{\mathrm{T}} \\ \vdots \\ \beta_L^{\mathrm{T}} \end{bmatrix}_{L \times n} \quad \text{and} \quad \mathbf{T} = \begin{bmatrix} \mathbf{t}_1^{\mathrm{T}} \\ \vdots \\ \mathbf{t}_P^{\mathrm{T}} \end{bmatrix}_{P \times n}.$$

Here, $\mathbf{H}$ is called the hidden layer output matrix of the SLFN. Thus, the system (5) becomes a linear model and the output weights can be analytically determined by finding a least-square solution of this linear system as follows:

$$\beta = \mathbf{H}^{\dagger}\mathbf{T} \tag{6}$$

where $\mathbf{H}^{\dagger}$ is the Moore–Penrose generalized inverse of matrix $\mathbf{H}$ [15]. Thus, ELM can be summarized as Algorithm 1.

Algorithm 1. ELM.

Input: a training set: $\{(\mathbf{x}_i, \mathbf{t}_i) | \mathbf{x}_i \in \mathbb{R}^m, \mathbf{t}_i \in \mathbb{R}^n, i = 1, \ldots, P\}$;
hidden node activation function: $g(x)$;
hidden node number: L.

Output β.

1 Assign arbitrary learning parameters of hidden nodes $\mathbf{a}_j$ and b_j, $1 \le j \le L$

2 Calculate the hidden layer output matrix $\mathbf{H}$ based on (5)

3 Calculate the output weights $\beta = \mathbf{H}^{\dagger}\mathbf{T}$.

Although ELM learning has been developed to work at a much faster learning speed with the higher generalization performance, it also has some drawbacks:

1. ELM is designed with the empirical risk minimization (ERM) principle and tends to generate an over-fitting model.
2. ELM provides weak control capacity and is less stable since it directly calculates the minimum norm least-square solutions.

3. Users have to choose the number of hidden nodes by trial-and-error and how to choose the optimal number of hidden nodes is still a problem.

2.2. Regularization methods

Multiple-linear regression is often used to investigate the relationship between the predictor variables and the response variable. Then both the prediction accuracy and the size of the model should be considered.

Considering the general setup for a single-output regression problem:

$$\mathbf{y} = \mathbf{H}\beta + \varepsilon \tag{7}$$

where $\mathbf{H}$ is the inputs of the problem, and it is a $P \times L$ matrix with $\mathbf{H} = [\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_i, \ldots, \mathbf{h}_P]^{\mathrm{T}}$, $\mathbf{h}_i = [h_{i1}, h_{i2}, \ldots, h_{iL}]^{\mathrm{T}}$. Here $\mathbf{y} = [y_1, y_2, \ldots, y_P]^{\mathrm{T}}$ is the actual output, $\beta = [\beta_1, \beta_2, \ldots, \beta_L]^{\mathrm{T}}$ is the regression weights, $\varepsilon = [\varepsilon_1, \varepsilon_2, \cdots, \varepsilon_P]^{\mathrm{T}}$ is the resident between target value and the actual output value. The traditional approach used to solve the above problem is the ordinary least square (OLS) estimates, which can be formulated as follows:

$$\widehat{\beta} = \arg\min_{\beta} \|\mathbf{y} - \mathbf{H}\beta\|_2^2 \tag{8}$$

where $\widehat{\beta} = \left[\widehat{\beta}_1, \widehat{\beta}_2, \ldots, \widehat{\beta}_L\right]^{\mathrm{T}}$ is the estimated regression weights. It is well-known that OLS often performs not well in terms of both the prediction accuracy and the model size [16]. Regularization techniques have been proposed to improve OLS.

There are some of the most widely used methods. They can be used to reflect the relationship between the hidden layer and the output layer of the ELM.

In general, L_1-norm, which is also called the least absolute shrinkage and selection operator (Lasso) [17], represents the most basic augmentation of the OLS solution. It shrinks some coefficients and sets others to zero. The Lasso estimate $\widehat{\beta}$ is defined by

$$\widehat{\beta} = \arg\min_{\beta} \left\{ \|\mathbf{y} - \mathbf{H}\beta\|_2^2 + \lambda \|\beta\|_1 \right\} \tag{9}$$

where λ is a positive regularization parameter and $\|\cdot\|_1$ denotes the L_1-norm. As λ increases, the number of nonzero components of $\widehat{\beta}$ decreases. In [18], a modified LARS algorithm is presented for variable selection and the Lasso can be effectively implemented through the use of this LARS.

Due to its nature of both continuous shrinkage and automatic variable selection simultaneously, the Lasso has shown success in many situations. But it has some limitations as noted by Zou and Hastie in [19]:

1. If there is a group of variables among which the pairwise correlations are very high, the lasso just selects only one variable from the group and does not care which one is selected.
2. For classical situation of $P > L$, if there are high correlations between predictors, it has been empirically observed that the prediction performance of the lasso is dominated by ridge regression [20].
3. In the situation of $P < L$, the Lasso selects at most P variables, which seems to be a limiting feature for a variable selection method.

Ridge regression minimizes the residual sum of squares subject to a bound on the L_2-norm of the coefficients [21], then the mathematic model of L_2-norm can be described as

$$\widehat{\beta} = \arg\min_{\beta} \left\{ \|\mathbf{y} - \mathbf{H}\beta\|_2^2 + \xi \|\beta\|_2^2 \right\} \tag{10}$$

where ξ is a L_2-norm regularization parameter. The above (10) can lead to a closed form solution as follows:

$$\widehat{\beta} = (\mathbf{H}^{\mathrm{T}}\mathbf{H} + \xi\mathbf{I})^{-1}\mathbf{H}^{\mathrm{T}}\mathbf{y} \tag{11}$$

where $\mathbf{I}$ is the identity matrix. L_2-norm can shrink coefficients and is more stable. It ensures a unique solution even when $L > P$ and leads to a grouping effect. However, it does not set any coefficients to 0, so it does not provide an easily interpretable model.

To overcome the drawbacks of L_1-norm and L_2-norm, both the L_1 penalty and the L_2 penalty are used in the same minimization problem. The mathematic model of this mixed penalties can be formulated as follows:

$$\widehat{\beta} = \arg\min_{\beta} \left\{ \|\mathbf{y} - \mathbf{H}\beta\|_2^2 + \lambda \|\beta\|_1 + \xi \|\beta\|_2^2 \right\} \tag{12}$$

where both λ and ξ are tuning parameters. The role of λ offers a variable selection through a sparse vector, ξ ensures a unique solution and leads to a grouping effect. In [22], the elastic net was proposed to solve (12).

The elastic net simultaneously does automatic variable selection and continuous shrinkage, and it can select group of correlated variables. It has been shown that the elastic net often outperforms the lasso in terms of prediction accuracy, while enjoying a similar sparsity of representation.

3. L_1-L_2-ELM model

3.1. Solution of the elastic net

The basic idea of solving the elastic net is to reduce the elastic net problem to an equivalent Lasso problem.

For data set $(\mathbf{y}, \mathbf{H})$ and (λ, ξ) defined in (12), an artificial data set $(\mathbf{y}^*, \mathbf{H}^*)$ is generated as follows:

$$\begin{cases} \mathbf{H}^*_{(P+L)\times L} = \dfrac{1}{\sqrt{(1+\xi)}} \begin{pmatrix} \mathbf{H} \\ \sqrt{\xi}\mathbf{I} \end{pmatrix} \\ \mathbf{y}^*_{(P+L)\times L} = \begin{pmatrix} \mathbf{y} \\ 0 \end{pmatrix} \end{cases} \tag{13}$$

Then the naive elastic net criterion can be written as

$$\widehat{\beta}^* = \arg\min_{\beta^*} \left\{ \|\mathbf{y}^* - \mathbf{H}^*\beta^*\|_2^2 + r\|\beta^*\|_1 \right\} \tag{14}$$

where $r = \lambda/\sqrt{(1+\xi)}$ and $\beta^* = \sqrt{1+\xi}\beta$. Thus, $\widehat{\beta}$ can be represented as follows:

$$\widehat{\beta} = \frac{1}{\sqrt{1+\xi}}\widehat{\beta}^* \tag{15}$$

However, the above solution may incur a double amount of shrinkage. In the first stage, for fixed L_2 penalty, the ridge regression shrinks the coefficients. Then the Lasso also shrinks the coefficients along the solution paths, and it depends on ξ. Thus the double shrinkage occurs, which may introduce unnecessary extra bias and do not help to reduce the variances [23].

Then the elastic net estimates as follows:

$$\widehat{\beta} = \sqrt{1+\xi}\,\widehat{\beta}^* \tag{16}$$

As we can see from (15) and (16), the following relationship can be obtained:

$$\widehat{\beta}(\text{in (16)}) = (1+\xi)\widehat{\beta}(\text{in (15)}) \tag{17}$$

Thus the elastic net avoids the ridge shrinkage effect by introducing a scaling factor $(1+\xi)$, while it still keeps the grouping effect feature of ridge regression. Hence, the solution of elastic net has been successfully transformed into the lasso problem. Then an

efficient algorithm called LARS-EN, was implemented to solve the elastic net solution paths for any fixed ξ [19].

3.2. Model selection based on information criteria

For each fixed ξ, the LARS-EN algorithm will produce the entire solution paths of the elastic net. Then a set of candidate models are obtained. The purpose of model selection is to select the optimal parameter λ and its corresponding solution path, which will determine the final model structure. In model selection, the Akaike information criterion (AIC) [24] and BIC were widely used. The AIC tends to select a model with more hidden nodes than necessary, which easily tends to an over-fitting model. Thus, the BIC is finally used for model selection to balance the accuracy and network size. The BIC for the kth candidate model is defined as follows:

$$\mathrm{BIC}(k) = \|\mathbf{y} - \mathbf{H}\beta(k)\|_2^2 + \log(P)\sigma M(k) \tag{18}$$

where $\beta(k)$ represents the kth candidate model, P is the number of samples, $M(k)$ is the number of non-zero elements of $\beta(k)$, σ is the residual variance of a low-bias model, and it can be defined as

$$\sigma = \frac{1}{P}\|\mathbf{y} - \mathbf{H}\mathbf{H}^{\dagger}\mathbf{y}\|_2^2 \tag{19}$$

where $\mathbf{H}^{\dagger}$ is the Moore–Penrose pseudo-inverse of $\mathbf{H}$.

3.3. L_1–L_2-ELM model

Both multiple-output regression and the multiple-class classification tasks can be implemented using a unified network model in the proposed algorithm.

For multiclass classification problem, it can be transformed into a multiple output regression problem. Assume a set of multiple-class training samples $(\mathbf{x}_i, \mathbf{t}_i)$ $(i = 1, 2, \ldots, P)$, $\mathbf{t}_i \in \{1, 2, \ldots, n\}$, each class label is expanded into a label vector of length n in accordance with the original ELM algorithm [25]. For example, in a training sample $(\mathbf{x}_i, \mathbf{t}_i)$, if the class label of $\mathbf{x}_i$ is q, the corresponding output vector of the n output nodes is $\mathbf{t}_i = [\overbrace{-1, \ldots, -1}^{q-1}, \underbrace{1, -1, \ldots, -1}_{n}]$. Namely, for a given testing sample **test**, the output node with the largest value indicates its predicted class label, and it can be formulated as follows:

$$\mathrm{label}(\mathbf{test}) = \arg\max_k \{\mathrm{y_test}_k\}, k \in \{1, 2, \ldots, n\} \tag{20}$$

where $\mathbf{y_test} = [\mathrm{y_}\mathbf{test}_1, \ldots, \mathrm{y_}\mathbf{test}_n]$ is the output vector for the testing sample **test**.

Then, for the regression problem with n output nodes, the proposed algorithm uses n separate elastic nets to generate the entire solution paths for each output node, then adopts the BIC to find the optimal candidate model. Thus, the output weights of ELM consist of all the optimal candidate model for each response variable. Finally, the proposed L_1–L_2-ELM algorithm can be summarized in Algorithm 2.

Algorithm 2. L_1–L_2-ELM.

Input: a training set: $\{(\mathbf{x}_i, \mathbf{t}_i) \mid \mathbf{x}_i \in \mathbb{R}^m, \mathbf{t}_i \in \mathbb{R}^n, i = 1, \cdots, P\}$;
hidden node activation function: $g(x)$;
the max hidden node number: L;
a fixed L_2 penalty term: ξ .

Output: β.

Assign arbitrary learning parameters of hidden nodes $\mathbf{a}_j$ and b_j, $1 \leqslant j \leqslant L$;
Calculate the hidden layer output matrix $\mathbf{H}$ based on (5);
for $1 \leqslant i \leqslant n$ **do**
  $\beta^i = \text{LARS-EN}\,(\mathbf{H}, \mathbf{y}(i), \xi)$, where $\mathbf{y}(i) = [t_{1i}, \cdots, t_{Pi}]^{\mathrm{T}}$;
  $\beta^i = (1 + \xi)\beta^i$;
  for $1 \leqslant k \leqslant \mathrm{size}(\beta^i)$ **do**
    Calculate the $\mathrm{BIC}(k)$ for every candidate model based on (18) and (19).
  end
  $k^* = \arg\min_k \{\mathrm{BIC}(k)\}_{k=1}^{\mathrm{size}(\beta^i)}$, where k^* is the index of the minimum value in vector **BIC**;
  $\beta^i_{\mathrm{optimal}} = \beta^i(k^*)$;
  $\beta = [\beta \quad \beta^i_{\mathrm{optimal}}]$.
end

4. Simulation results and discussion

4.1. Experimental setup

To verify the effectiveness of the proposed algorithm L_1–L_2-ELM, the Lorenz chaotic time series [26] and nine data sets from the UCI machine learning repository [27] have been used to test this algorithm, and we compare it with the basic ELM, L_2-ELM [25], and the OP-ELM [28], which is a regularized ELM using the LARS algorithm, i.e., a L_1 penalty. The activation function used in these four algorithms is a simple sigmoidal function $g(x) = 1/(1 + e^{-x})$. The number of hidden nodes $L = 100$ is set equally for ELM, L_2-ELM, OP-ELM, and L_1–L_2-ELM. In the implementation of L_2-ELM and L_1–L_2-ELM, it is found that the performance of these two algorithms is

influenced by the constraint factor ξ. Then, in our experiments, ξ is chosen from the range $\{2^{-24}, 2^{-23}, \ldots, 2^{24}, 2^{25}\}$.

For each benchmark data set, we first randomly permute it, and then almost two-thirds of the whole data set are used as the training set, one-third is used as the testing set. And 50 trials have been conducted for all the algorithms and the best performance as well as the standard deviations (DEV) is recorded. Nodes required by L_1–L_2-ELM in each data set can be obtained by calculating the average of the numbers of selected neurons for each response node.

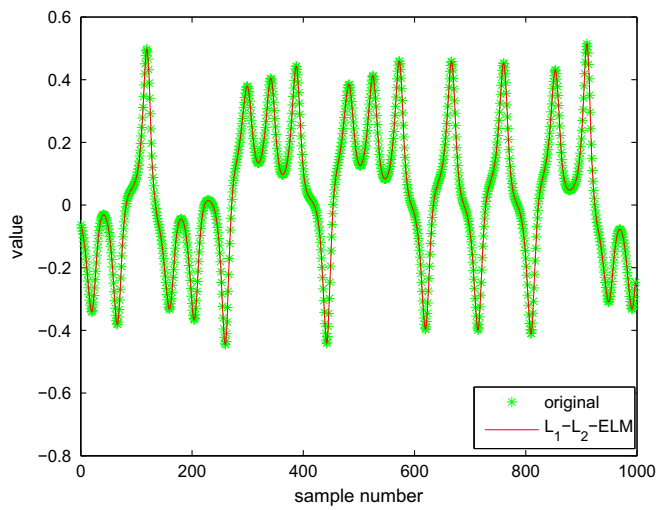

Fig. 1. Prediction of the L_1–L_2-ELM for Lorenz function.

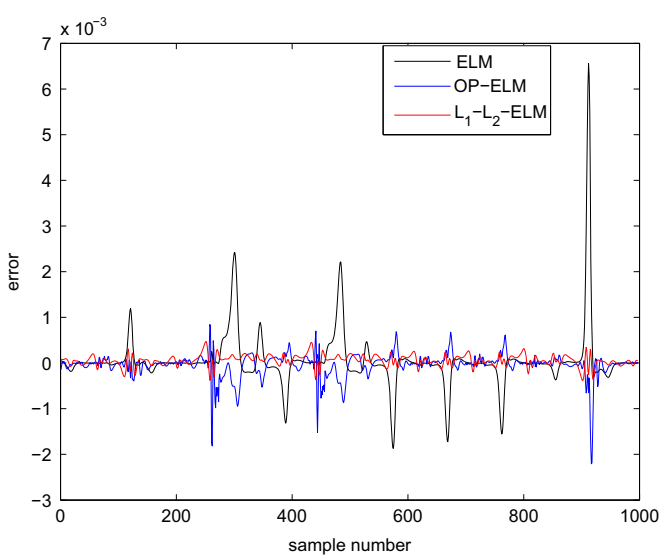

Fig. 2. Comparison of prediction error for Lorenz.

4.2. Artificial case: prediction of the Lorenz chaotic time series

In this example, the data is derived from the Lorenz System, which is given by three time-delay differential systems

$$\begin{cases} \dfrac{dx(t)}{dt} = \sigma(y(t) - x(t)) \\ \dfrac{dy(t)}{dt} = rx(t) - y(t) - x(t)z(t) \\ \dfrac{dz(t)}{dt} = -bz(t) + x(t)y(t) \end{cases} \tag{21}$$

where $x(t)$, $y(t)$, and $z(t)$ are the values of time series at time t. A typical choice for the parameters in (21) are $\sigma = 16$, $b = 4$, and $r = 45.92$.

The data set is constructed by using four-order Runge–Kutta method with the initial value as $x(0) = -1$, $y(0) = 0$, and $z(0) = 1$, the step size is chosen as 0.01. In order to extract the dynamic characteristic of the Lorenz system to predict $x(t+1)$, the embedded data vector is chosen as

$$\mathbf{X}(t) = (x(t), x(t-\tau), \ldots, x(t-(M-1)\tau)) \tag{22}$$

where for the phase space reconstruction, τ is the delay time, M is the embedded dimension, and $\tau = 1$, $M = 3$. Then we can get 1500 samples. The former 500 samples are used as the training set, and the other 1000 samples are used as the testing set.

The prediction results are shown in Figs. 1 and 2. Fig. 1 shows that the prediction values of L_1–L_2-ELM algorithm are in agreement with the true values. Fig. 2 shows the prediction error of each sampling point for three algorithms. It is clear that the L_1–L_2-ELM has a better prediction performance than original ELM and OP-ELM.

4.3. Real-world regression problems

The specifications of the 4 real-world benchmark data sets [27] are listed in Table 1. As we can see from Table 2, the proposed algorithm is better than the OP-ELM and the original ELM in terms of the testing root mean square error (RMSE) and the DEV of testing accuracy, which means that the proposed algorithm has a better prediction performance and is more robust than these two algorithms. The ELM is often trapped between approximation and generalization, and it is only based on empirical risk minimization. In this way it tends to result an over-fitting model. Thus, the basic ELM may have high RMSE and DEV on some testing sets. The OP-ELM introduces L_1 penalty to ELM in nature, and prunes some hidden nodes in accordance with their usefulness. Thus, compared to basic ELM, the OP-ELM can keep a good RMSE on all the testing data sets. But it does not consider the high correlations between hidden nodes in ELM, then it may generate a suboptimal model. Based on L_1 norm, we also introduce L_2 penalty to ELM, which can encourage a group effect, and groups of correlated hidden nodes can be selected. Therefore, our proposed L_1–L_2-ELM benefits from ridge regression and the tendency towards sparse solution of the Lasso, and it can generate a more suitable NN structure than OP-ELM. By introducing the L_2 penalty to ELM, L_2-ELM can adjust the

Table 1
Information of the regression data sets.

Data sets	Attributes	Samples	
		Training	Testing
Abalone	8	2784	1393
Machine CPU	6	139	70
Servo	4	110	57
Stocks	9	633	317

proportion of empirical risk and structure risk, which will make the model more robust, but it still does not shrink the coefficient of irrelevant variables to zero. Thus it could not provide an interpretable model, and it still may have problems when encountering irrelevant data. From Table 2, it can be found that the L_2-ELM achieves better generalization performance than ELM, but there are some problems on some testing sets, while L_1–L_2-ELM remains good RMSE and DEV on all testing sets.

Table 3 shows the computational time for these four algorithms on regression data sets. From Table 3, we find that basic ELM and L_2-ELM are faster than OP-ELM and L_1–L_2-ELM when training the NN, which is due to that the basic ELM and L_2-ELM get the solution of output weights by directly solving a linear equation, while the OP-ELM and L_1–L_2-ELM spend much time on selecting some excellent nodes from hidden layer, and then they get the appropriate NN structure. However, the OP-ELM and L_1–L_2-ELM are both faster than ELM and L_2-ELM for testing the NN due to their pruned NN structure.

4.4. Real-world classification problems

The specifications of the 5 real-world classification data sets [27] are listed in Table 4. The comparison results are shown in Table 5. In Table 5, the L_1–L_2-ELM can achieve a higher success

Table 2
Comparison of RMSE and DEV in ELM, L_2-ELM, OP-ELM, and L_1–L_2-ELM on the regression data sets.

Methods	Datasets	RMSE		DEV		Nodes/ξ
		Training	Testing	Training	Testing	
ELM	Abalone	1.9994	2.1491	0.0047	0.0238	100/–
L_2-ELM		2.0254	2.1276	0.0032	0.0132	$100/2^{-15}$
OP-ELM		2.0755	2.1596	0.0268	0.0220	60/–
L_1–L_2-ELM		2.0405	2.1238	0.0159	0.0117	$45/2^{-20}$
ELM	Machine CPU	137.7941	235.7793	7.1300	4.1434e+11	100/–
L_2-ELM		132.9674	143.0855	7.9255	15.4769	$100/2^{-5}$
OP-ELM		19.0124	75.8023	14.5489	27.8916	65/–
L_1–L_2-ELM		28.9849	35.0180	1.2623	3.1088	$66/2^{-4}$
ELM	Servo	0.0424	3.5394	0.0155	1.4206	100/–
L_2-ELM		0.2073	0.6480	0.0146	0.0572	$100/2^{-14}$
OP-ELM		0.2556	0.8962	0.1022	0.1219	60/–
L_1–L_2-ELM		0.2072	0.6597	0.0201	0.0694	$65/2^{-14}$
ELM	Stocks	1.4396	1.6541	0.4608	0.7261	100/–
L_2-ELM		1.5710	1.5896	0.6002	0.6903	$100/2^{-18}$
OP-ELM		0.9450	1.0636	0.0731	0.0812	75/–
L_1–L_2-ELM		0.7376	0.9642	0.0370	0.0515	$89/2^{-21}$

Table 3
Comparison of computational time for ELM, L_2-ELM, OP-ELM, and L_1–L_2-ELM algorithms on the regression data sets.

Methods	Datasets	Time (s)	
		Training	Testing
ELM	Abalone	0.1404	0.0468
L_2-ELM		0.1560	0.0468
OP-ELM		3.3658	0.0134
L_1–L_2-ELM		5.1553	0.0310
ELM	Machine CPU	0.0624	0.0468
L_2-ELM		0.0624	0.0468
OP-ELM		0.1555	0.0025
L_1–L_2-ELM		0.2953	0.0049
ELM	Servo	0.0624	0.0312
L_2-ELM		0.0624	0.0468
OP-ELM		0.1436	0.0018
L_1–L_2-ELM		0.4440	0.0059
ELM	Stocks	0.0468	0.0312
L_2-ELM		0.0624	0.0468
OP-ELM		0.3915	0.0041
L_1–L_2-ELM		1.2146	0.0111

Table 4
Information of the classification data sets.

Data Sets	Attributes/classes	Samples	
Iris	4/3	100	50
Wine	13/3	118	60
Landsat satellite	36/6	4290	2145
Vehicle Silhouettes	18/4	564	282
MAGIC gamma telescope	11/2	12,680	6340

Table 5
Comparison of success rate and DEV in ELM, L_2-ELM, OP-ELM, and L_1–L_2-ELM on the classification data sets.

Methods	Data sets	Success rate		DEV		Nodes/ξ
		Training	Testing	Training	Testing	
ELM	Iris	1.0000	0.7800	0.0000	0.1017	100/–
L_2-ELM		0.9800	0.9800	0.0071	0.0104	$100/2^{-1}$
OP-ELM		0.9800	0.9600	0.0167	0.0341	30/–
L_1–L_2-ELM		1.0000	0.9800	0.0090	0.0081	$7/2^{-12}$
ELM	Wine	0.8475	0.8333	0.0665	0.0847	100/–
L_2-ELM		0.8814	0.8667	0.0746	0.0817	$100/2^{-9}$
OP-ELM		1.0000	0.9667	0.0089	0.0315	51/–
L_1–L_2-ELM		1.0000	0.9833	0.0043	0.0247	$26.3/2^{-12}$
ELM	Landsat satellite	0.7431	0.7235	0.0318	0.0305	100/–
L_2-ELM		0.7459	0.7343	0.0286	0.0280	$100/2^{2}$
OP-ELM		0.8422	0.8177	0.0053	0.0068	100/–
L_1–L_2-ELM		0.8606	0.8443	0.0035	0.0040	$31.5/2^{-8}$
ELM	Vehicle Silhouettes	0.6507	0.6099	0.0367	0.0424	100/–
L_2-ELM		0.6365	0.6135	0.0371	0.0414	$100/2^{0}$
OP-ELM		0.7518	0.6667	0.0234	0.0235	75/–
L_1–L_2-ELM		0.8121	0.7801	0.0157	0.0175	$26.2/2^{-9}$
ELM	MAGIC gamma telescope	0.7565	0.7511	0.0063	0.0060	100/–
L_2-ELM		0.7553	0.7598	0.0053	0.0059	$100/2^{0}$
OP-ELM		0.8374	0.8328	0.0056	0.0059	71/–
L_1–L_2-ELM		0.8358	0.8336	0.0031	0.0018	$55/2^{-24}$

Table 6
Comparison of computational time for ELM, L_2-ELM, OP-ELM, and L_1–L_2-ELM algorithms on the classification data sets.

Methods	Data sets	Time (s)	
		Training	Testing
ELM	Iris	0.0624	0.0468
L_2-ELM		0.0624	0.0468
OP-ELM		0.1772	0.0024
L_1–L_2-ELM		0.9724	0.0050
ELM	Wine	0.0624	0.0468
L_2-ELM		0.0624	0.0468
OP-ELM		0.1502	0.0014
L_1–L_2-ELM		1.8912	0.0059
ELM	Landsat satellite	0.2184	0.0312
L_2-ELM		0.2340	0.0312
OP-ELM		9.9610	0.0492
L_1–L_2-ELM		29.6272	0.0375
ELM	Vehicle Silhouettes	0.0468	0.0312
L_2-ELM		0.0780	0.0468
OP-ELM		0.3866	0.0048
L_1–L_2-ELM		2.7966	0.0098
ELM	MAGIC Gamma Telescope	0.9828	0.1092
L_2-ELM		0.4836	0.0936
OP-ELM		20.3506	0.1367
L_1–L_2-ELM		123.2136	0.1464

rate for testing samples. And the DEV of testing success rate is much lower than the other algorithms, which means that the proposed algorithm can achieve more accurate and stable classification performance. For multi-class classification problem, we transform it into multi-output regression problems. Considering the nature of the elastic net and different correlations between hidden nodes and each output node, we establish a separate model for each output node respectively. Thus our method may prune different hidden nodes for different output nodes, then it will generate a more appropriate classification model. Namely, we will introduce zeros for some entries in all columns of the output weights. For OP-ELM, it explores the relativity of each hidden node to all the output nodes, thus it will introduce zeros for all entries in some rows of output weights, which may weak the different correlations between some hidden nodes and some personalized output nodes.

Moreover, it is obvious that the computational training time of L_1–L_2-ELM is slower than OP-ELM, L_2-ELM, and basic ELM in Table 6. Due to the fact that the elastic net algorithm is directly employed to solve the L_1 and L_2 problems on a special case with one output, we have to establish a separate model for each output node in ELM to solve multi-output problems, which will cost much time for NN training. And it can easily deduce that the training time of our method will become longer as the number of the output nodes in NN increases. Although heavier computation is required, our method can get a more appropriate NN structure.

5. Conclusion

Data analysis plays a huge guidance role for making future plans in practical applications. In this paper, a novel algorithm called L_1–L_2-ELM was proposed as an effective technology in data analysis. It can deal with multiple-output regression and multiple-class classification problems in a unified framework. The proposed algorithm adds a L_2 penalty term based on the L_1-norm regularized ELM, thus it can obtain the good properties of the Lasso (L_1-norm) and the ridge regression (L_2-norm). The proposed algorithm firstly generates all the candidate models, then uses the BIC to find the optimal solution of the learning model. Generally speaking, for W multiple-output applications, W separate elastic nets need to be used to solve the problem. Even though our proposed algorithm is slower than the other three algorithms when training NN, simulation results have shown that L_1–L_2-ELM can achieve a better generalization performance and variable selection ability than the ELM, L_2-ELM, and OP-ELM, especially in multiple-class applications. Meanwhile the L_1–L_2-ELM is more robust than the other three algorithms.

References

[1] B. Brown, M. Chui, J. Manyika, Are you ready for the era of 'big data', McKinsey Q. 4 (2011) 24–35.
[2] D. Yu, L. Deng, Efficient and effective algorithms for training single-hidden-layer neural networks, Pattern Recognit. Lett. 33 (5) (2012) 554–558.
[3] S. Ding, C. Su, J. Yu, An optimizing BP neural network algorithm based on genetic algorithm, Artif. Intell. Rev. 36 (2) (2011) 153–162.
[4] X. Luo, X. Chang, H. Liu, A Taylor based localization algorithm for wireless sensor network using extreme learning machine, IEICE Trans. Inf. Syst. E 97D (10) (2014) 2652–2659.
[5] G.-B. Huang, Q. Zhu, C. Siew, Extreme learning machine: theory and applications, Neurocomputing 70 (1) (2006) 489–501.
[6] X. Luo, X. Chang, A novel data fusion scheme using grey model and extreme learning machine in wireless sensor networks, Int. J. Control Autom. Syst. 13 (3) (2015) 539–546.
[7] P. Horata, S. Chiewchanwattana, K. Sunat, Robust extreme learning machine, Neurocomputing 102 (2013) 31–44.
[8] W. Deng, Q. Zheng, L. Chen, Regularized extreme learning machine, in: Proceedings of IEEE Symposium on Computational Intelligence and Data Mining, Nashville, 2009, pp. 389–395.
[9] H.-J. Rong, Y.-S. Ong, A.-H. Tan, Z. Zhu, A fast pruned-extreme learning machine for classification problem, Neurocomputing 72 (1) (2008) 359–366.
[10] J.M. Martínez-Martínez, P. Escandell-Montero, E. Soria-Olivas, J.D. Martín-Guerrero, R. Magdalena-Benedito, J. Gómez-Sanchis, Regularized extreme learning machine for regression problems, Neurocomputing 74 (17) (2011) 3716–3721.
[11] Y. Miche, A. Sorjamaa, P. Bas, O. Simula, C. Jutten, A. Lendasse, OP-ELM: optimally pruned extreme learning machine, IEEE Trans. Neural Netw. 21 (1) (2010) 158–162.
[12] C. de Mol, E. de Vito, L. Rosasco, Elastic-net regularization in learning theory, J. Complex. 25 (2) (2009) 201–230.
[13] K.P. Burnham, D.R. Anderson, Multimodel inference understanding AIC and BIC in model selection, Sociol. Methods Res. 33 (2) (2004) 261–304.
[14] Q. He, T. Shang, F. Zhuang, Z. Shi, Parallel extreme learning machine for regression based on MapReduce, Neurocomputing 102 (2013) 52–58.
[15] G.-B. Huang, D.H. Wang, Y. Lan, Extreme learning machines: a survey, Int. J. Mach. Learn. Cybern. 2 (2) (2011) 107–122.
[16] R. Tibshirani, Regression shrinkage and selection via the lasso, J. R. Stat. Soc. Ser. B (Methodological) (1996) 267–288.
[17] L. Jacob, G. Obozinski, J.-P. Vert, Group lasso with overlap and graph lasso, in: Proceedings of the 26th Annual International Conference on Machine Learning, Montreal, 2009, pp. 433–440.
[18] B. Efron, T. Hastie, I. Johnstone, R. Tibshirani, Least angle regression, Ann. Stat. 32 (2) (2004) 407–499.
[19] H. Zou, T. Hastie, Regularization and variable selection via the elastic net, J. R. Stat. Soc.: Ser. B (Stat. Methodol.) 67 (2) (2005) 301–320.
[20] L. Breiman, Heuristics of instability and stabilization in model selection, Ann. Stat. 24 (6) (1996) 2350–2383.
[21] A.E. Hoerl, R.W. Kennard, Ridge regression: biased estimation for nonorthogonal problems, Technometrics 12 (1) (1970) 55–67.
[22] T. Zhou, D. Tao, X. Wu, Manifold elastic net: a unified framework for sparse dimension reduction, Data Min. Knowl. Discov. 22 (3) (2011) 340–371.
[23] Q. Li, N. Lin, The Bayesian elastic net, Bayesian Anal. 5 (1) (2010) 151–170.
[24] H. Bozdogan, Model selection and Akaike's information criterion (AIC): the general theory and its analytical extensions, Psychometrika 52 (3) (1987) 345–370.
[25] G.-B. Huang, H. Zhou, X. Ding, R. Zhang, Extreme learning machine for regression and multiclass classification, IEEE Trans. Syst. Man Cybern.: Part B 42 (2) (2012) 513–529.
[26] D. Li, M. Han, J. Wang, Chaotic time series prediction based on a novel robust echo state network, IEEE Trans. Neural Netw. Learn. Syst. 23 (5) (2012) 787–799.
[27] A. Asuncion, D. Newman, UCI machine learning repository, 2007. 〈http://archive.ics.uci.edu/ml/〉.
[28] A. Grigorievskiy, Y. Miche, A.-M. Ventelä, E. Séverin, A. Lendasse, Long-term time series prediction using OP-ELM, Neural Netw. 51 (2014) 50–56.

Xiong Luo received the Ph.D. degree from Central South University, Changsha, China, in 2004. From 2005 to 2006, he was with the Department of Computer Science and Technology, Tsinghua University, China, as a Postdoctoral Fellow. From 2012 to 2013, he was with the School of Electrical, Computer and Energy Engineering, Arizona State University, USA, as a Visiting Scholar. He currently works as an Associate Professor in the Department of Computer Science and Technology, School of Computer and Communication Engineering, University of Science and Technology Beijing, China. His research interests include computational intelligence, intelligent control, and machine learning.

Xiaohui Chang is currently working toward the Master's degree at the Department of Computer Science and Technology, School of Computer and Communication Engineering, University of Science and Technology Beijing, China. Her research interests include machine learning and computational intelligence.

Xiaojuan Ban received the Ph.D. degree from University of Science and Technology Beijing, China, in 2003. She currently works as a Professor in the Department of Computer Science and Technology, School of Computer and Communication Engineering, University of Science and Technology Beijing, China. Her research interests include artificial intelligence, machine learning, and computer animation.